Méthodes de programmation modulaire

Avantages d'une programmation modulaire

En première on a introduit les fonctions en réponse à la répétition de code au sein d'un programme. Cependant, la répétition des fonctions entre programmes est encore possible. Une solution est de regrouper ces fonctions et classes dans un **module**. On **décompose** ainsi une application en module(s) qui peuvent être **développés indépendamment** et **réutilisés** dans d'autres applications.

Par ailleurs, cette organisation limite ou **supprime les doublons** de code, facilitant ainsi la maintenance (*un bug est corrigé une fois*).

Enfin, la programmation modulaire permet d'une certaine façon de **cacher des détails d'implémentation**, l'interaction utilisateur-module se faisant par le biais de l'interface.

Il s'agira dans l'activité 1, de se placer en temps qu'utilisateur de modules en se référant à leur API, puis en temps que concepteur d'un module simple, dans la suivante.

Activité 1: traitement de données au format JSON

Présentation rapide du format json

Le fichier `json` est un fichier texte contenant de l'information structurée, très utilisé sur internet. Cette structuration est assurée par deux types d'éléments:

- des paires clé/valeur (qui rappelle les dictionnaires de python);
- de listes ordonnées de valeurs.

Voir [json sur wikipedia](). Voici un exemple de données au format `json`:

```
{
    "menu": {
        "id": "file",
        "value": "File",
        "popup": {
            "menuitem": [
                { "value": "New", "onclick": "CreateNewDoc()" },
                { "value": "Open", "onclick": "OpenDoc()" },
                { "value": "Close", "onclick": "CloseDoc()" }
            ]
        }
    }
}
```

Découvrir la structure d'un fichier réel

Le site openweathermap permet d'obtenir des informations météo dans beaucoup de villes du monde entier. Pour obtenir le temps dans une ville donnée, il faut fournir un identifiant unique ou la paire lattitude/longitude. Openweathermap propose un fichier `json` contenant ces informations. Ce fichier est disponible en version compressée, ici.

> **ATTENTION**
>
> Ne pas double-cliquer pour ouvrir avec l'application par défaut, le fichier a une taille de 33 Mo et plus de 2 millions de lignes!!

On préfèrera un traitement en ligne de commande pour voir la structure du fichier. On rappelle que l'utilisation de la ligne de commande est possible dans une cellule `jupyter notebook`, il suffit de préfixer la commande avec le caractère `!`.

Enfin, pour plus de précisions sur une commande `bash`, ne pas hésiter à consulter le manuel (exemple: `man sed`).

```
# Nombre de lignes
!cat ./json/city.list.json | wc -l
```

```
2095791
```

```
# 2 premières lignes concernant la Réunion
!cat ./json/city.list.json | grep -n "RE" | head -n 2
```

```
238646:    "country": "RE",
238656:    "country": "RE",
```

```
# Affichage de 20 lignes autour du 1er résultat
!sed -n "238642,+20p" ./json/city.list.json
```

```
  {
    "id": 935146,
    "name": "Les Trois-Bassins",
    "state": "",
    "country": "RE",
    "coord": {
      "lon": 55.299999,
      "lat": -21.1
    }
  },
  {
    "id": 935210,
    "name": "Salazie",
    "state": "",
    "country": "RE",
    "coord": {
      "lon": 55.539001,
      "lat": -21.02734
    }
  },
  {
```


Objectifs

Le but de cette première activité est d'extraire de ce fichier uniquement les informations concernant les villes de la Réunion et de les sauvegarder dans un fichier au format json. Le fichier obtenu devrait être nettement moins lourd et plus facilement utilisable (une soixantaine d'entrées au lieu de plus de 200 000 dans le fichier original).

Rappels de 1re

Pour importer un module, on peut utiliser la syntaxe `import mon_super_module` ; ce qui nous obligera à préfixer chaque appel de fonction par `mon_super_module.ma_fonction()`. On peut *raccourcir* les appels en utilisant un alias lors de l'import: `import mon_super_module as msp`, on appelle alors `msp.ma_fonction()`.
Enfin, on peut cibler une fonction précise à importer. Dans ce cas, on utilisera la syntaxe: `from mon_super_module import ma_fonction` et l'appel se fera directement par `ma_fonction()` sans préfixe.

Travail à réaliser

A faire

Prendre connaissance de l'api du module json, en ligne sur python.org ou à défaut dans une cellule de Jupyter en utilisant les fonctions `dir` et `help`.

```python
import json


dir(json)
```

A faire

Quelles sont les deux fonctions du module `json` susceptibles de nous intéresser en vue d'une ouverture et d'une écriture de fichiers json ? Pour obtenir de l'aide sur une fonction présente dans un module on utilise la fonction `help` (par exemple `help(json.xxxx)` où `xxxx` est le nom d'une fonction du module `json`).

```python
#Réponse
```

A faire

On va extraire que les informations concernant les villes de la Réunion. Quelle fonction doit-on utiliser pour transformer le fichier `city.list.json` en une structure utilisable en python ? Réaliser cette opération en complétant le code ci-dessous.

```python
import json

# Création d'un objet fichier f lié à city.list.json
with open('./json/city.list.json', 'r') as f:
    #A compléter (supprimer la ligne 'pass')
    pass
```

A faire

Quelle est le type de la structure obtenue et combien d'entrées possède-t-elle?

```python
#Réponse
```

A faire

Afficher la 1ere entrée.

```
#Réponse
```

A faire

Proposer un code permettant de sauver les entrées qui ne concernent que la Réunion dans un tableau.

```
#Réponse
villes_RE = []
#A compléter
```

A faire

Combien d'entrée possède ce nouveau tableau ?

```
#Réponse
```

A faire

Quelle fonction devra-t-on utilser pour transformer ce tableau et le sauver dans un fichier json ?

```
#Réponse
```

A faire

Proposer un code qui réalise la sauvergarde du tableau `villes_RE` dans un fichier json `villes_RE.json`.

```python
#Réponse

# Création d'un objet fichier f lié à villes_RE.json
with open("./json/villes_RE.json", "w") as f:
    #à compléter ici
    pass
```

```python

```